

projet pssm

Job 01

Infrastructure de base et règles de sécurité

VM FTP

```
Debian GNU/Linux 12 toutpourelareussiteFTP tty1
toutpourelareussiteFTP login: toutpourelareussiteftp
Password:
Linux toutpourelareussiteFTP 6.1.0-53-amd64 #1 SMP
The programs included with the Debian GNU/Linux system are
the exact distribution terms for each program are described
individual files in /usr/share/doc/*/copyright.
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the
permitted by applicable law.
```

VM SQL

```
Debian GNU/Linux 12 toutpourelareussiteMariadb tty1
toutpourelareussiteMariadb login: toutpourelareussitemariadb
Password:
Linux toutpourelareussiteMariadb 6.1.0-53-amd64 #1 SMP PREEMPT
The programs included with the Debian GNU/Linux system are
the exact distribution terms for each program are described
individual files in /usr/share/doc/*/copyright.
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the
permitted by applicable law.
Last login: Mon Sep 28 12:15:27 CEST 2026 on tty1
toutpourelareussitemariadb@toutpourelareussiteMariadb:~$
```

VM Web

```
Debian GNU/Linux 12 toutpourelareussiteWeb tty1
toutpourelareussiteWeb login: toutpourelareussiteweb
Password:
Linux toutpourelareussiteWeb 6.1.0-53-amd64 #1 SMP PREEMPT
The programs included with the Debian GNU/Linux system are
the exact distribution terms for each program are described
individual files in /usr/share/doc/*/copyright.
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the
permitted by applicable law.
Last login: Tue Sep 15 13:09:58 CEST 2026 on tty1
toutpourelareussiteweb@toutpourelareussiteWeb:~$
```

Job 02

Préparation de la VM Debian sans interface graphique

```
sudo adduser monitor
```

```
usermod -aG sudo monitor
```

VM Web

```
sudo apt install -y nginx apache2-utils
```

Outils MySQL/MariaDB

```
sudo apt install -y mariadb-server  
mysql_secure_installation
```

Client FTP

```
sudo apt install -y proftpd
```

réseau La Plateforme

Job 02

Configurer l'authentification par clé SSH pour monitor

```
ssh-keygen -t ed25519 -C "monitor_key"
```

```
mkdir -p /home/monitor/.ssh  
chmod 700 /home/monitor/.ssh
```

Ajouter ta clé publique dans id_ed25519.pub

```
nano /home/monitor/.ssh/id_ed25519.pub
```

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDQ0zr537+0ykK+8FIKdtPRT1rqBB8QxbtHmPkpa
```

```
chmod 644 /home/monitor/.ssh/id_ed25519.pub  
chown -R monitor:monitor /home/monitor/.ssh
```

```
-rw----- 1 monitor monitor 419 17 sept. 10:47 id_ed25519  
-rw-r--r-- 1 monitor monitor 106 17 sept. 10:47 id_ed25519.pub
```

Job 03

Script ssh_login.py

```
import paramiko

hote = "10.60.5.147"
utilisateur = "monitor"
cle_privée = "/home/monitor/.ssh/id_ed25519"
commande = "ls -la"

ssh = paramiko.SSHClient()
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())

print("Tentative de connexion en cours")

ssh.connect(hostname=hote, username=utilisateur, key_filename=cle_privée)
print("Connexion SSH réussie")

stdin, stdout, stderr = ssh.exec_command(commande)

resultat = stdout.read().decode()
print(resultat)
ssh.close()
```

pip install paramiko mysql-connector-python

```
(venv) root@serverclient:~# python3 ~/psmm/ssh_login.py
Tentative de connexion en cours
Connexion SSH réussie
total 32
drwx----- 4 monitor monitor 4096 28 sept. 10:30 .
drwxr-xr-x 4 root root 4096 16 sept. 14:22 ..
-rw----- 1 monitor monitor 351 28 sept. 10:56 .bash_history
-rw-r--r-- 1 monitor monitor 220 16 sept. 14:22 .bash_logout
-rw-r--r-- 1 monitor monitor 3526 16 sept. 14:22 .bashrc
drwx----- 3 monitor monitor 4096 28 sept. 10:30 .config
-rw-r--r-- 1 monitor monitor 807 16 sept. 14:22 .profile
drwx----- 2 monitor monitor 4096 17 sept. 11:11 .ssh
-rw-r--r-- 1 monitor monitor 0 23 sept. 11:32 .sudo_as_admin_successful
```

Job 04 —

Script ssh_login_sudo.py

```
import paramiko

hote = "10.60.5.147"
utilisateur = "monitor"
cle_privée = "/home/monitor/.ssh/id_ed25519"
mdp_sudo = "root"

commande = "sudo apt update"

ssh = paramiko.SSHClient()
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())

ssh.connect(hostname=hote, username=utilisateur, key_filename=cle_privée)
print("Connexion SSH réussie")

stdin, stdout, stderr = ssh.exec_command(commande, get_pty=True)

stdin.write(mdp_sudo + "\n")
stdin.flush()

resultat = stdout.read().decode()
erreurs = stderr.read().decode()

print("\nRésultat de la commande sudo :", commande)
print(resultat)
ssh.close()
```

```
root@serverclient:~# source ~/psmm/venv/bin/activate
(venv) root@serverclient:~# python3 ~/psmm/ssh_login_sudo.py
Connexion SSH réussie

Résultat de la commande sudo : sudo apt update
root
[sudo] Mot de passe de monitor :
Atteint :1 http://deb.debian.org/debian bookworm InRelease
Atteint :2 http://deb.debian.org/debian bookworm-updates InRelease
Atteint :3 http://security.debian.org/debian-security bookworm-security InRelease
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
1 paquet peut être mis à jour. Exécutez « apt list --upgradable » pour le voir.

(venv) root@serverclient:~#
```

Job 05 — Connexion à MariaDB/MySQL

Script ssh_mysql.py

```
import paramiko

hote = "10.60.5.147"
utilisateur = "monitor"
cle_privée = "/home/monitor/.ssh/id_ed25519"
mdp_sudo = "root"

commande = "sudo mariadb -e 'SELECT VERSION(), NOW()';"

ssh = paramiko.SSHClient()
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())

ssh.connect(hostname=hote, username=utilisateur, key_filename=cle_privée)
print("Connexion SSH au serveur MariaDB réussie")

stdin, stdout, stderr = ssh.exec_command(commande, get_pty=True)

stdin.write(mdp_sudo + "\n")
stdin.flush()

resultat = stdout.read().decode()
erreurs = stderr.read().decode()

print(resultat)

ssh.close()
```

```
(env) root@serverclient:~# python3 ~/psmm/ssh_mysql.py
Connexion SSH au serveur MariaDB réussie
root
[sudo] Mot de passe de monitor :
+-----+-----+
| VERSION() | NOW() |
+-----+-----+
| 10.11.18-MariaDB-0+deb12u1 | 2026-10-01 14:54:13 |
+-----+-----+

(env) root@serverclient:~#
```

Job 06 — Erreurs d'authentification MariaDB

Créer la base de données

```
CREATE DATABASE psmm_db;
```

Créer la table des erreurs MySQL

```
USE psmm_db;
```

```
CREATE TABLE erreurs_mysql (
  id INT AUTO_INCREMENT PRIMARY KEY,
  date_heure DATETIME NOT NULL,
  compte VARCHAR(50) NOT NULL,
  ip VARCHAR(45) NOT NULL
);
```

```
Database changed
MariaDB [psmm_db]> DESCRIBE erreurs_mysql;
+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+
| id | int(11) | NO | PRI | NULL | auto_increment |
| date_heure | datetime | NO | | NULL | |
| compte | varchar(50) | NO | | NULL | |
| ip | varchar(45) | NO | | NULL | |
+-----+-----+
4 rows in set (0.013 sec)
```

```
MariaDB [psmm_db]>
```

Créer un utilisateur MariaDB dédié pour nos scripts

```
CREATE USER 'caleb'@'%' IDENTIFIED BY 'root';
```

Donne-lui tous les droits sur cette base

```
GRANT ALL PRIVILEGES ON psmm_db.* TO 'caleb'@'%';
```

```
FLUSH PRIVILEGES;
```

Autoriser les connexions distantes sur MariaDB

```
sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

```
# These groups are read by MariaDB server.
# Use it for options that only the server (but not clients) should see
# This is read by the standalone daemon and embedded servers
[server]

# This is only for the mysqld standalone daemon
[mysqld]
log_warnings = 2
log_error = /var/log/mysql/error.log
#
# * Basic Settings
#
user = mysql
pid-file = /run/mysqld/mysqld.pid
basedir = /usr
datadir = /var/lib/mysql
tmpdir = /tmp

# Broken reverse DNS slows down connections considerably and name resolve is
# safe to skip if there are no "host by domain name" access grants
skip-name-resolve

# Instead of skip-networking the default is now to listen only on
# localhost which is more compatible and is not less secure.
bind-address = 0.0.0.0
```

ssh_mysql_error.py

nano ~/psmm/ssh_mysql_error.py

```
import mysql.connector
import paramiko

ssh = paramiko.SSHClient()
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
ssh.connect(
    hostname='10.60.5.147',
    username='monitor',
    key_filename='/home/monitor/.ssh/id_ed25519',
)

cmd = "sudo journalctl -u mariadb --no-pager | grep 'Access denied for user'"
stdin, stdout, stderr = ssh.exec_command(cmd, get_pty=True)
stdin.write('root\n')
stdin.flush()

lignes = stdout.read().decode().splitlines()
ssh.close()
print('Logs récupérés avec succès')

bdd = mysql.connector.connect(
    host='10.60.5.147',
    user='caleb',
    password='root',
    database='psmm_db',
)
curseur = bdd.cursor()

for ligne in lignes:
    if 'Access denied for user' in ligne:
        partie_date = ligne.split('Warning')[0].strip().split()
        date_heure = f'{partie_date[-3]} {partie_date[-2]}'

        reste = ligne.split('Access denied for user')[1]
        compte = reste.split('@')[0]
        ip = reste.split(':')[1].split(' ')[0]

        requete = (
            "INSERT INTO erreurs_mysql (date_heure, compte, ip) VALUES (%s, %s, %s)"
        )
        curseur.execute(requete, (date_heure, compte, ip))
        print(f'Ajouté : {date_heure} | {compte} | {ip}')

bdd.commit()
curseur.close()
bdd.close()
```

```
(venv) root@serverclient:~# python3 ~/psmm/ssh_mysql_error.py
Logs récupérés avec succès
Ajouté : 2026-09-23 14:06:00 | popi | 10.60.5.147
Ajouté : 2026-09-23 14:06:36 | BBC | 10.60.5.147
Ajouté : 2026-09-23 14:42:39 | fauxuser | 10.60.5.199
Enregistrement en base terminé !
(venv) root@serverclient:~#
```

Job 07

Créer la table SQL erreurs_ftp

CREATE TABLE erreurs_ftp

```
Database changed
MariaDB [psmm_db]> DESCRIBE erreurs_ftp;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id     | int(11) | NO | PRI | NULL | auto_increment |
| date_heure | datetime | NO | | NULL | |
| compte | varchar(50) | NO | | NULL | |
| ip     | varchar(45) | NO | | NULL | |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.004 sec)
```

pour enregistrer les authentifications :

sudo nano /etc/proftpd/proftpd.conf

```
GNU nano 7.2 /etc/proftpd/proftpd.conf
#
# We want 'welcome.msg' displayed at login, and '.message' displayed
# in each newly chdir'd directory.
# DisplayLogin welcome.msg
# DisplayChdir .message
#
# Limit WRITE everywhere in the anonymous chroot
# <Directory */
# <Limit WRITE
# DenyAll
# </Limit>
# </Directory>
#
# Uncomment this if you're brave.
# # <Directory incoming>
# # # Unmask 022 is a good standard umask to prevent new files and dirs
# # # (second param) from being group and world writable.
# # Umask 022 022
# # <Limit READ WRITE>
# # DenyAll
# # </Limit>
# # <Limit STOR>
# # AllowAll
# # </Limit>
# # </Directory>
#
# Include other custom configuration files
# !! Please note, that this statement will read /all/ file from this subdir,
# i.e. backup files created by your editor, too !!!
# Eventually create file patterns like this: /etc/proftpd/conf.d/*.conf
#
ExtendedLog /var/log/proftpd/auth.log AUTH auth
```

```

import paramiko
import mysql.connector

ssh = paramiko.SSHClient()
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
ssh.connect(
    hostname="10.60.5.145",
    username="monitor",
    key_filename="/home/monitor/.ssh/id_ed25519"
)

cmd = "sudo grep 'USER' /var/log/proftpd/proftpd.log | grep -E 'no such user'"
stdin, stdout, stderr = ssh.exec_command(cmd, get_pty=True)
stdin.write('root\n')
stdin.flush()

lignes = stdout.read().decode().splitlines()
ssh.close()

bdd = mysql.connector.connect(
    host="10.60.5.147",
    user="caleb",
    password="root",
    database="psmm_db"
)
curseur = bdd.cursor()

for ligne in lignes:
    if 'USER' in ligne and 'from ' in ligne:
        date_brute = ligne.split()[0]
        heure_brute = ligne.split()[1].split(',')[0]
        date_heure = f'{date_brute} {heure_brute}'
        compte = ligne.split('USER ')[1].split(':')[0].strip()
        ip = ligne.split('from ')[1].split(':')[0].strip()
        requete = "INSERT INTO erreurs_ftp (date_heure, compte, ip) VALUES (%s, %s, %s)"
        curseur.execute(requete, (date_heure, compte, ip))
        print(f"Ajouté : {date_heure} | Compte: {compte} | IP: {ip}")

bdd.commit()
curseur.close()
bdd.close()

```

```

(venv) root@serverclient:~# python3 ~/psmm/ssh_ftp_error.py
Ajouté : 2026-10-01 16:14:57 | Compte: l'huile | IP: 10.60.5.199
Ajouté : 2026-10-01 16:15:54 | Compte: bombassdu75 | IP: 10.60.5.199
Ajouté : 2026-10-01 16:16:08 | Compte: caleb | IP: 10.60.5.199
(venv) root@serverclient:~#

```

Job 08

nano ~/psmm/ssh_web_error.py

```

(venv) root@serverclient:~# python3 ~/psmm/ssh_web_error.py
Logs Web récupérés avec succès.
Ajouté : 2026-10-02 10:09:58 | Compte: moi | IP: 10.60.5.199
Ajouté : 2026-10-02 10:10:35 | Compte: moi | IP: 10.60.5.199
Ajouté : 2026-10-02 10:10:42 | Compte: moi | IP: 10.60.5.199
(venv) root@serverclient:~#

```

```

import mysql.connector
import paramiko

ssh = paramiko.SSHClient()
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
ssh.connect(
    hostname="10.60.5.146",
    username="monitor",
    key_filename="/home/monitor/.ssh/id_ed25519",
)

cmd = "sudo grep 'user' /var/log/nginx/error.log | grep -E 'password mismatch'"
stdin, stdout, stderr = ssh.exec_command(cmd, get_pty=True)
stdin.write('root\n')
stdin.flush()

lignes = stdout.read().decode().splitlines()
ssh.close()
print('Logs Web récupérés avec succès.')

bdd = mysql.connector.connect(
    host="10.60.5.147",
    user="caleb",
    password="root",
    database="psmm_db",
)
curseur = bdd.cursor()

for ligne in lignes:
    if 'user' in ligne and 'client: ' in ligne:
        morceaux = ligne.split()
        date_brute = morceaux[0].replace('/', '-')
        heure_brute = morceaux[1]
        date_heure = f'{date_brute} {heure_brute}'
        compte = ligne.split('user ')[1].split(':')[0].strip()
        ip = ligne.split('client: ')[1].split(',')[0].strip()
        requete = (
            "INSERT INTO erreurs_web (date_heure, compte, ip) VALUES (%s, %s, %s)"
        )
        curseur.execute(requete, (date_heure, compte, ip))
        print(f"Ajouté : {date_heure} | Compte: {compte} | IP: {ip}")

bdd.commit()
curseur.close()
bdd.close()

```

Job 10

Créer le dossier de sauvegarde

`mkdir -p ~/psmm/backups`

`nano ~/psmm/ssh_cron_backup.py`

```
from datetime import datetime
from pathlib import Path
import subprocess

hote_bdd = "18.68.5.147"
user_bdd = "caleb"
mdp_bdd = "root"
nom_bdd = "psmm_db"

dossier_backup = Path.home() / "psmm" / "backups"
dossier_backup.mkdir(parents=True, exist_ok=True)

horodatage = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
fichier_sortie = dossier_backup / f"backup_psmm_{horodatage}.sql"

commande = [
    "mysqldump",
    "-h",
    hote_bdd,
    "-u",
    user_bdd,
    "-p",
    f"{mdp_bdd}",
    nom_bdd,
]

with open(fichier_sortie, "w") as f:
    subprocess.run(commande, stdout=f, check=True)

print(f"Sauvegarde réussie : {fichier_sortie.name}")

sauvegardes = sorted(dossier_backup.glob("backup_psmm_*.sql"))

while len(sauvegardes) > 7:
    plus_vieux = sauvegardes.pop(0)
    plus_vieux.unlink()
    print(f"Ancienne sauvegarde supprimée : {plus_vieux.name}")

print(f"Nombre de sauvegardes conservées : {len(sauvegardes)}")
```

Job 10

Créer le dossier de sauvegarde

`mkdir -p ~/psmm/backups`

`nano ~/psmm/ssh_cron_backup.py`

```
from datetime import datetime
from pathlib import Path
import subprocess

hote_bdd = "18.68.5.147"
user_bdd = "caleb"
mdp_bdd = "root"
nom_bdd = "psmm_db"

dossier_backup = Path.home() / "psmm" / "backups"
dossier_backup.mkdir(parents=True, exist_ok=True)

horodatage = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
fichier_sortie = dossier_backup / f"backup_psmm_{horodatage}.sql"

commande = [
    "mysqldump",
    "-h",
    hote_bdd,
    "-u",
    user_bdd,
    "-p",
    f"{mdp_bdd}",
    nom_bdd,
]

with open(fichier_sortie, "w") as f:
    subprocess.run(commande, stdout=f, check=True)

print(f"Sauvegarde réussie : {fichier_sortie.name}")

sauvegardes = sorted(dossier_backup.glob("backup_psmm_*.sql"))

while len(sauvegardes) > 7:
    plus_vieux = sauvegardes.pop(0)
    plus_vieux.unlink()
    print(f"Ancienne sauvegarde supprimée : {plus_vieux.name}")

print(f"Nombre de sauvegardes conservées : {len(sauvegardes)}")
```

0 <3 * * * /home/monitor/psmm/venv/bin/python3 /home/monitor/psmm/ssh_cron_backup.py >> /home/monitor/psmm/cron_backup.log 2>&1

Job 11

```
root@serverweb:~# mysql -e "SELECT * FROM psmm_db.system_status;"
```

id	date_heure	serveur	cpu_percent	ram_percent	disk_percent
7	2026-10-02 11:26:24	ftp	50	18.0962	11
8	2026-10-02 11:26:24	web	50	18.1538	11
9	2026-10-02 11:26:24	mariaadb	50	24.9359	12

```
(venv) root@serverclient:~# nano ~/psmm/ssh_system_status.py
(venv) root@serverclient:~# python3 ~/psmm/ssh_system_status.py
OK ftp :) CPU: 50.0% | RAM: 18.1% | DISQUE: 11.0%
OK web :) CPU: 50.0% | RAM: 18.2% | DISQUE: 11.0%
OK mariadb :) CPU: 58.0% | RAM: 24.9% | DISQUE: 12.0%
(venv) root@serverclient:~#
```

```
nano ~/psmm/ssh_system_status.py
```

Job 12

```
GNU nano 7.2 /root/psmm/ssh_system_mail.py
from datetime import datetime
from email.message import EmailMessage
from pathlib import Path
import smtplib
import time
import mysql.connector
import paramiko

SEUIL_CPU = 70.0
SEUIL_RAM = 80.0
SEUIL_DISK = 90.0

FICHIER_TIMESTAMP = Path.home() / "psmm" / "dernier_mail.txt"

SMTP_SERVERUE = "smtp.gmail.com"
SMTP_PORT = 465
EXPEDITEUR = "olagorj83@gmail.com"
MOT_DE_PASSE = "kvppeegnjjjobgand"
DESTINATAIRE = "olagorj83@gmail.com"
```

```
if cpu > SEUIL_CPU:
    alertes.append(f"- {nom_serveur} : CPU a {cpu:.1f}% (seuil: {SEUIL_CPU}%)*")
if ram > SEUIL_RAM:
    alertes.append(f"- {nom_serveur} : RAM a {ram:.1f}% (seuil: {SEUIL_RAM}%)*")
if disk > SEUIL_DISK:
    alertes.append(f"- {nom_serveur} : Disque a {disk:.1f}% (seuil: {SEUIL_DISK}%)*")
```

```
if alertes:
    maintenant = time.time()
    pour_emailver = True

    if FICHERE_TIMESTAMP.exists():
        dernier_email = float(FICHERE_TIMESTAMP.read_text()).strip()
        if maintenant - dernier_email < 3600:
            temps_attente_nuis = FICHERE_TIMESTAMP.read_text().strip()
            temps_attente_nuis = float(temps_attente_nuis) * (maintenant - dernier_email) / 60
            pour_emailver = (temps_attente_nuis < 1)
            temps_attente_nuis = 1
        else:
            pour_emailver = False

    if pour_emailver:
        print('on mail d\'alerte')
        corps_mail = ''
        # on liste les messages existants (date_actuelle) \n\n
        * '\n', join(alertes)
```

```
(env) root@serverclient:~# python3 ~/psmm/ssh_system_mail.py
Envoi du mail d'alerte
Mail envoyé
(env) root@serverclient:~# python3 ~/psmm/ssh_system_mail.py
Alerte présente mais ignorée : un mail a déjà été envoyé il y a moins d'une heure (attente: 59 min).
```

Job 14

```
cmd_update = "sudo apt update && sudo apt upgrade -y"
cmd_reboot = "test -f /var/run/reboot-required && echo 'OUI' || echo 'NON'"
```

```
for nom, ip in serveurs.items():
    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    ssh.connect(hostname=ip, username=utilisateur_ssh, key_filename=cle_ssh)

    stdin, stdout, stderr = ssh.exec_command(cmd_update, get_pty=True)
    stdin.write(mdp_sudo + "\n")
    stdin.flush()
    stdout.channel.recv_exit_status()

    stdin, stdout, stderr = ssh.exec_command(cmd_reboot)
    besoin_reboot = stdout.read().decode().strip()

    ssh.close()

    if besoin_reboot == "OUI":
        statut = "Mise a jour reussi redemarrage "
    else:
        statut = "Aucun redemarrage necessaire"

    print(f"{nom} : {statut}")
    lignes_rapport.append(f"- {nom} ({ip}) : {statut}")
```

[MAJ SYSTEME] Rapport des mises a jour de securite

olagojr83@gmail.com

À moi

Rapport des mises a jour systeme du 2026-10-02 12:02:56 :

- ftp (10.60.5.145) : Aucun redemarrage necessaire
- web (10.60.5.146) : Aucun redemarrage necessaire
- mariadb (10.60.5.147) : Aucun redemarrage necessaire

```
(venv) root@serverclient:~# python3 ~/psmm/ssh_system_maj.py
ftp : Aucun redemarrage necessaire
web : Aucun redemarrage necessaire
mariadb : Aucun redemarrage necessaire
Envoi du mail
(venv) root@serverclient:~#
```